

Quick Reference Guide

Module 3: Artificial Intelligence Technology Fundamentals: Deep Learning

Learning Outcomes:

- Identify examples of applications that employ deep learning algorithms.
- Identify the components of neural networks.
- Calculate the gradient descent of a neural network.
- Implement deep learning techniques in Python.
- Analyze simulated neural networks.
- Calculate convolutional neural networks' outputs.
- Categorize neural network algorithms.

Introduction to Deep Learning

Deep learning uses neural networks to model complex data patterns, achieving high accuracy in tasks like image recognition and natural language processing.

Key Concepts and Frameworks

- Machine learning methods
 - Supervised uses labelled data
 - Unsupervised uses unlabelled data
 - Semi-supervised combines both
- Parametric vs. non-parametric
 - Parametric methods have fixed parameters
 - Non-parametric methods have flexible structures
- Probabilistic models
 - Incorporate uncertainty, such as Bayesian method

Neural Networks Overview

- Multi-layer perceptrons (MLPs): The basic structure consisting of an input layer, hidden layers, and an output layer. This is used for classification and regression tasks.
- Convolutional neural networks (CNNs): This is specialized for image processing and uses convolutional layers to capture spatial hierarchies.
- Recurrent neural networks (RNNs): This is designed for sequence data (e.g., time series, text) and is capable of retaining information from previous inputs.

Examples of Applications Employing Deep Learning Algorithms

- Image recognition and processing: CNNs for facial recognition and medical imaging.
- Natural language processing (NLP): RNNs for language translation and sentiment analysis.
- Speech recognition: Deep neural networks for voice assistants and transcription services.
- Autonomous vehicles: CNNs and RNNs for object detection and decision making.

Learning to Cure Cancer

- Breast cancer statistics: 2 million cases per year, 13% risk in US women; early diagnosis lowers mortality; excessive screening has a 19% false positive rate.
- AI personalized screening (Tempo): Developed by Professor Regina Barzilay; uses diverse data; detects cancer 4.5 months earlier; reduces mammograms by 25%.
- Multimodal processing: Integrates various data types for better early detection and risk prediction; applies to other cancers.
- Research findings: Multimodal methods enhance diagnosis accuracy; valuable for other cancers; need for open multimodal databases.

Calculating Performance Metrics for Classification Models

Steps to calculate the accuracy, F1 score, precision, and recall based on the Confusion matrix:

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Steps:

- Accuracy
 - Add true positives (TP) and true negatives (TN).
 - Divide by the total number of predictions (TP + TN + FP + FN)
- Precision
 - Divide true positives (TP) by the sum of true positives (TP) and false positives (FP).
- Recall
 - Divide true positives (TP) by the sum of true positives (TP) and false negatives (FN).
- F1 score
 - Calculate the harmonic mean of precision and recall:
 - Multiply precision and recall.
 - Multiply the result by 2.
 - Divide by the sum of precision and recall.

Artificial Neural Networks: Components, Function, and Learning with Gradient Descent

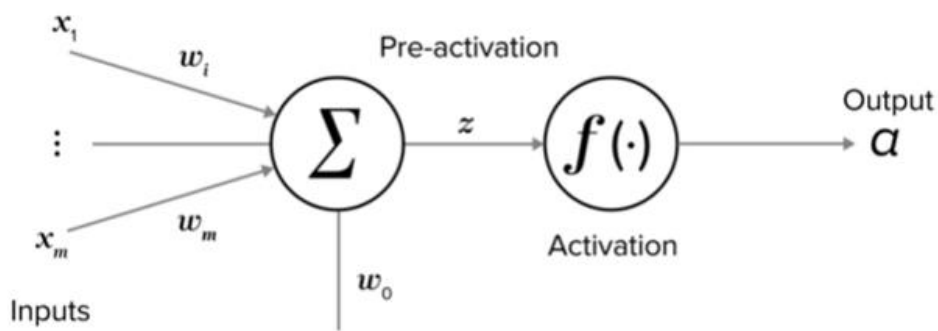
An artificial neural network (ANN) mimics the brain by using connected neurons to process inputs and produce outputs. Weights are adjusted during training to improve pattern recognition.

Key components:

- Input layer

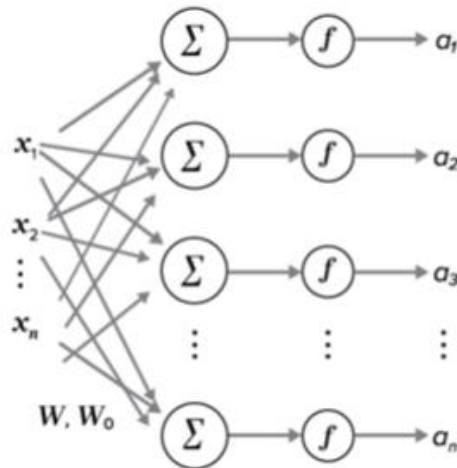
- Hidden layers
- Weights
- Activation function
- Output layer
- Bias
- Loss function
- Optimizer
- Examples
- Layers connectivity

Neural network is a mathematical expression that is expressed as



Potentially nonlinear activation function f produces output $a = f(z)$

For multilayer neural network expression is expressed as



Mathematical notation: Arrange the weight multiplications and summations in the matrix form $Z = WX + W_0$

Calculation of Gradient Descent Optimization for Artificial Neurons

- Express in terms of the loss $L(\cdot)$ associated with the prediction errors over n training points

$$J(\theta, \theta_0) = \sum_{i=1}^n L_{NN}(x^{(i)}, y^{(i)})$$

- Calculate the gradient with respect to the model coefficients

$$\nabla_{\theta} J(\theta, \theta_0) = g(\theta, \theta_0, D_{train})$$

- Modify the coefficients in a downhill direction of the gradient, with learning rate η

$$\theta^{(t+1)} = \theta^t - \eta \nabla_{\theta} J$$

Overview of Perceptrons and Activation Functions

- Perceptrons: Is a basic unit of neural networks, mimics brain neurons.
- Function: Processes binary inputs, computes weighted sum, adds bias, applies activation function to produce output.
- Usage: Suitable for binary classification with linearly separable data.

Activation Functions

Selecting the right activation function is key to effective learning and convergence in deep learning models.

- Sigmoid: Used in binary classification output.

$$\text{Sigmoid: } \sigma(z) = \frac{1}{1+e^{-z}}$$

- Hyperbolic tangent (tanh): Used in the hidden layers for centered data.

$$\text{Hyperbolic tangent: } \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- ReLU: Used in the hidden layers in deep networks.

Rectified linear unit: $\text{ReLU}(z) = \max(0, z)$

- Leaky ReLU: Used as an alternative to ReLU.
- Softmax: Used in multi-class classification output.

Types of Perceptron

Single-Layer Perceptron (SLP): Is a neural network with one layer of output nodes.

- Benefits: Handles linearly separable data, enables simple binary classification, is efficient and memory-friendly, and is easy to interpret.
- Limitation: Struggles with linearly non-separable problems, such as XOR

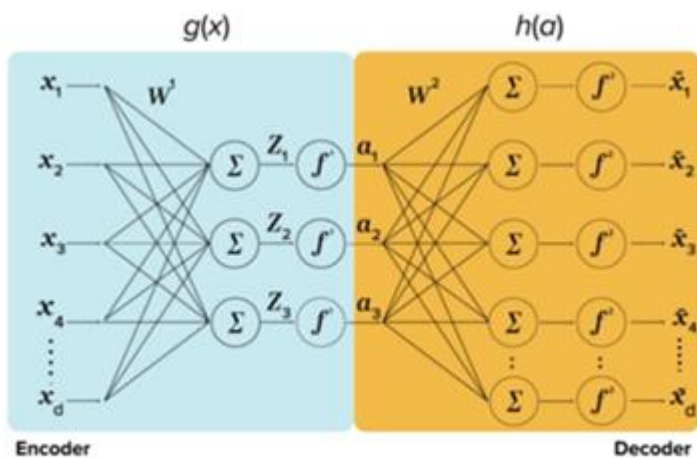
Multi-Layer Perceptron (MLP): Is a type of artificial neural network with multiple layers of nodes.

- Structure of MLP:
 - Input layer
 - Hidden layers (one or more)
 - Output layer
- Connections:
 - Fully connected nodes between adjacent layers
 - Each connection has an associated weight
- Activation functions:
 - Sigmoid

- Tanh
- Rectified linear unit (ReLU)
- Capabilities:
 - Classification
 - Regression
 - Pattern recognition
- Training algorithm:
 - Backpropagation

Autoencoder Overview

An autoencoder is an unsupervised neural network for dimensionality reduction and data denoising, comprising an encoder to compress data and a decoder to reconstruct it.



TensorFlow: Is an open-source framework by Google for building and deploying machine learning models, supporting GPUs/TPUs and multiple programming languages.

Steps for Building an Autoencoder with TensorFlow:

1. Prepare and preprocess the dataset.
2. Define the encoder and decoder architecture.
3. Set a loss function to measure reconstruction error.
4. Choose an optimizer to minimize the loss.
5. Train the autoencoder with the dataset.
6. Evaluate performance with a validation/test set.

7. Use the trained model for encoding and reconstruction of new data.

Evolution of a Neural Network

To adjust the parameters, you can experiment with neural networks by changing the number of neurons, learning rate, and activation to observe how these changes affect the behavior of the neural network. Go to [Neural Network Playground](#) explore the different behaviors of a neural network by tuning various parameters. Based on your experiment you can draw conclusions from different simulated neural networks.

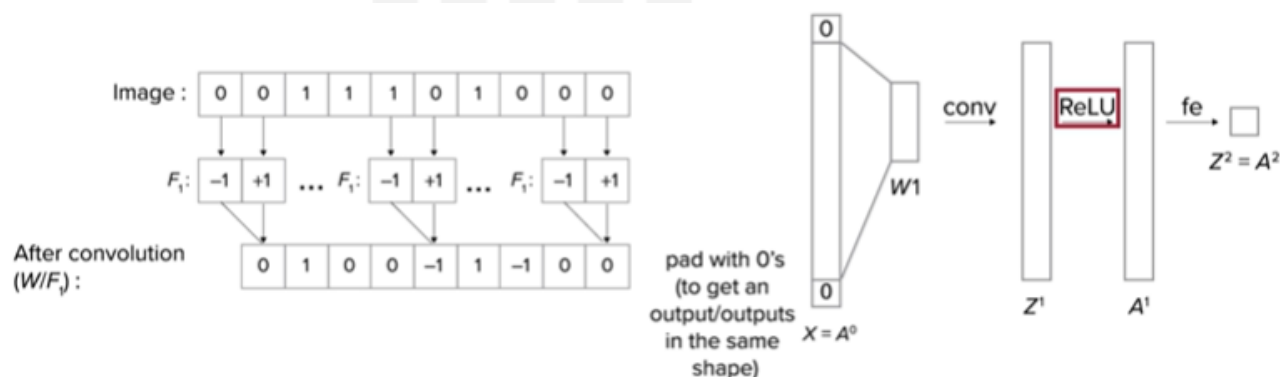
Convolutional Neural Networks (CNNs)

CNNs are deep neural networks specialized for image tasks like recognition and classification.

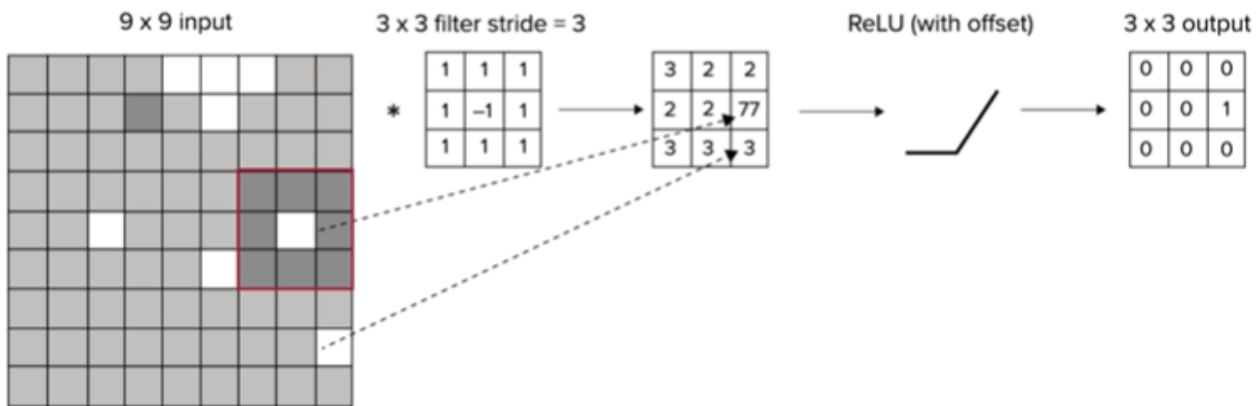
Bottom of form

This handles spatial data like images using spatial locality. Examples of spatial data include:

- 1D convolution: Applies a weighted filter to a sequence.



- 2D convolution: Filters slide over images to extract features, followed by ReLU activation.



Learning: CNNs train to adjust filter weights for feature detection.

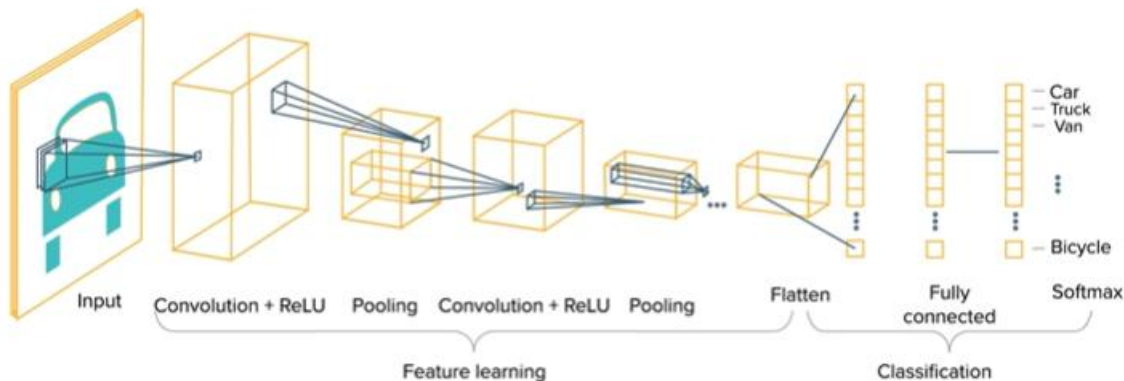
Layers

- Input layer: Receives the image or volume of images.
- Convolutional layer: Applies filters to extract features from the input.
- Activation layer: Uses ReLU to introduce nonlinearity.
- Pooling layer: Down samples feature maps using max or average pooling.
- Flattening layer: Converts feature maps into a 1D vector.
- Fully connected layer: Connects all neurons for classification or regression.
- Normalization layer: (Optional) Normalizes inputs for better training.
- Output layer: Produces final predictions, often with softmax for classification.

Deep Neural Networks (DNNs)

DNNs are neural networks with many layers, designed to perform complex tasks like image classification.

The structure and components of deep neural networks



Key aspects include:

- Architecture: Convolutional, activation, pooling, and fully connected layers.
- Training: Uses gradient backpropagation; requires powerful computation.
- Challenges: Overfitting, extensive data needs, complex hyperparameters.
- Transfer learning: Fine-tunes pre-trained networks for new tasks.

Recurrent Neural Networks (RNNs)

RNNs are designed for sequence data, like speech recognition and language translation.

Key aspects include:

- Architecture: Consist of an input, hidden state (h), and output (o) at each time step with feedback loops.
- Training: Uses supervised learning and gradient descent; faces vanishing/exploding gradients.
- Advanced variants: LSTMs manage past information with gating functions. GRUs simplify LSTMs, offering faster training.

Types of RNNs:

- One-to-one
- One-to-many
- Many-to-one
- Many-to-many

Popular RNN Architectures:

- Bidirectional RNN (BRNN): Processes input sequences in both forward and backward directions.
- Gated recurrent unit (GRU): Combines forget and input gates into a single update gate, with an additional reset gate.
- Long short-term memory (LSTM): Designed to overcome the vanishing gradient problem, suitable for learning long-range dependencies.

